UCS503P - Software Engineering Project

Sortify: AI-Powered Waste Identification and Disposal Guidance

Prototype Stage Report

BE, Computer Science and Engineering Department

Submitted by

1024160015 Daksh Aggarwal, CSED
1024160002 Mankirat Singh, CSED
1024160028 Saket Gatyan, CSED

Faculty Advisor

Dr. Jeelani Asif

Prototype Stage Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **2**
- 1.1 Background and Context . . . 2
- 1.2 Problem Statement . . . 2
- 1.3 Prototype Objectives . . . 3

2 Methodology and Design **4**
- 2.1 System Design Reference . . . 4
- 2.2 Functional Flow . . . 4
- 2.3 Use Cases Represented in the Prototype . . . 7
- 2.4 Data Model Reference . . . 7
- 2.5 Technical Stack Implemented . . . 9

3 Implemented Prototype Features **10**
- 3.1 Application Navigation and Layout . . . 10
- 3.2 Waste Catalog and Guide . . . 10
- 3.3 Scan Demo . . . 12
- 3.4 Result and Disposal Guidance . . . 13
- 3.5 History and Activity . . . 13
- 3.6 Profile and Local Data Settings . . . 13
- 3.7 Scripted Assistant . . . 16
- 3.8 Admin Demo . . . 16

4 Results and Evaluation **18**
- 4.1 Prototype Status . . . 18
- 4.2 Implemented Versus Not Yet Implemented . . . 18
- 4.3 Current Evaluation Limits . . . 18

5 Conclusion **20**
- 5.1 Summary of Work . . . 20
- 5.2 Next Work Based on Proposal Scope . . . 20

Chapter 1

Introduction

1.1 Background and Context

Sortify is a mobile-first waste identification and disposal guidance system. The higher-order goal stated for the project is to reduce uncertainty at the point where a user decides how to dispose of an item. The system is intended to help users move from an unidentified waste item to actionable disposal instructions instead of only providing a raw classification label.

The current prototype focuses on demonstrating the user-facing workflow through a React Native mobile application. It presents a working local demo of scanning, sample prediction results, a searchable waste guide, saved history, feedback submission, a scripted assistant, profile settings, and a demo administrative interface. The prototype does not yet connect to a deployed backend service or a trained image-classification model.

1.2 Problem Statement

Users often encounter everyday items whose disposal route is unclear. The project proposal identifies common pain points such as uncertainty about waste categories, fragmented disposal information, incorrect segregation, lack of actionable guidance, and location-dependent disposal rules.

The prototype addresses this problem at the interface level by making disposal guidance visible immediately after a demo scan or manual catalog selection. It also shows how the user can save results, review them later, ask item-specific questions, and submit corrections when a result appears wrong.

1.3 Prototype Objectives

The prototype-stage work completed so far demonstrates the following implemented objectives:

1. Build a mobile application shell with navigation between Sortify's main user-facing screens.
2. Implement a local waste catalog covering the proposed waste streams: recyclable, organic, general waste, e-waste, and hazardous waste.
3. Demonstrate a scan-to-result workflow using selectable demo samples and simulated confidence values.
4. Provide actionable disposal steps, caution notes, and local guidance text for catalog items.
5. Support local history, profile settings, feedback, assistant messages, and catalog edits using device-local storage.

Chapter 2

Methodology and Design

2.1 System Design Reference

The project proposal defines Sortify as a system combining a mobile application, ML-based identification, a structured waste knowledge base, location-aware guidance, history, feedback, administrative tools, and a conversational assistant. At the prototype stage, the implemented system is a local mobile demo that models these interactions without claiming live backend, database, or ML deployment.

2.2 Functional Flow

The implemented demo follows the core proposal workflow in prototype form:

1. The user opens the app and navigates to the scan screen.
2. The user either selects a sample item or uploads/captures a photo.
3. For uploaded or captured photos, the prototype asks the user to choose the demo result; the actual image is not sent to a backend.
4. The app simulates prediction processing and opens a result screen with a demo confidence value.
5. The result screen shows item category, material, disposal steps, caution notes where available, and local guidance text.
6. The user can save the item, mark it as sorted, submit a correction, ask the assistant about the item, or return to the guide.

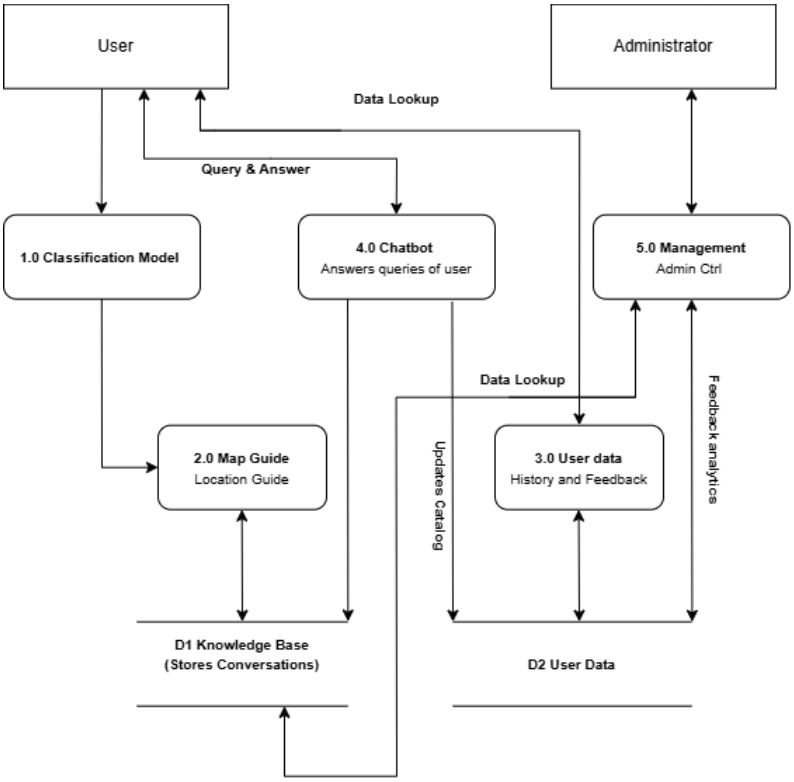


Figure 2.1: Level-1 Data Flow Diagram for Sortify

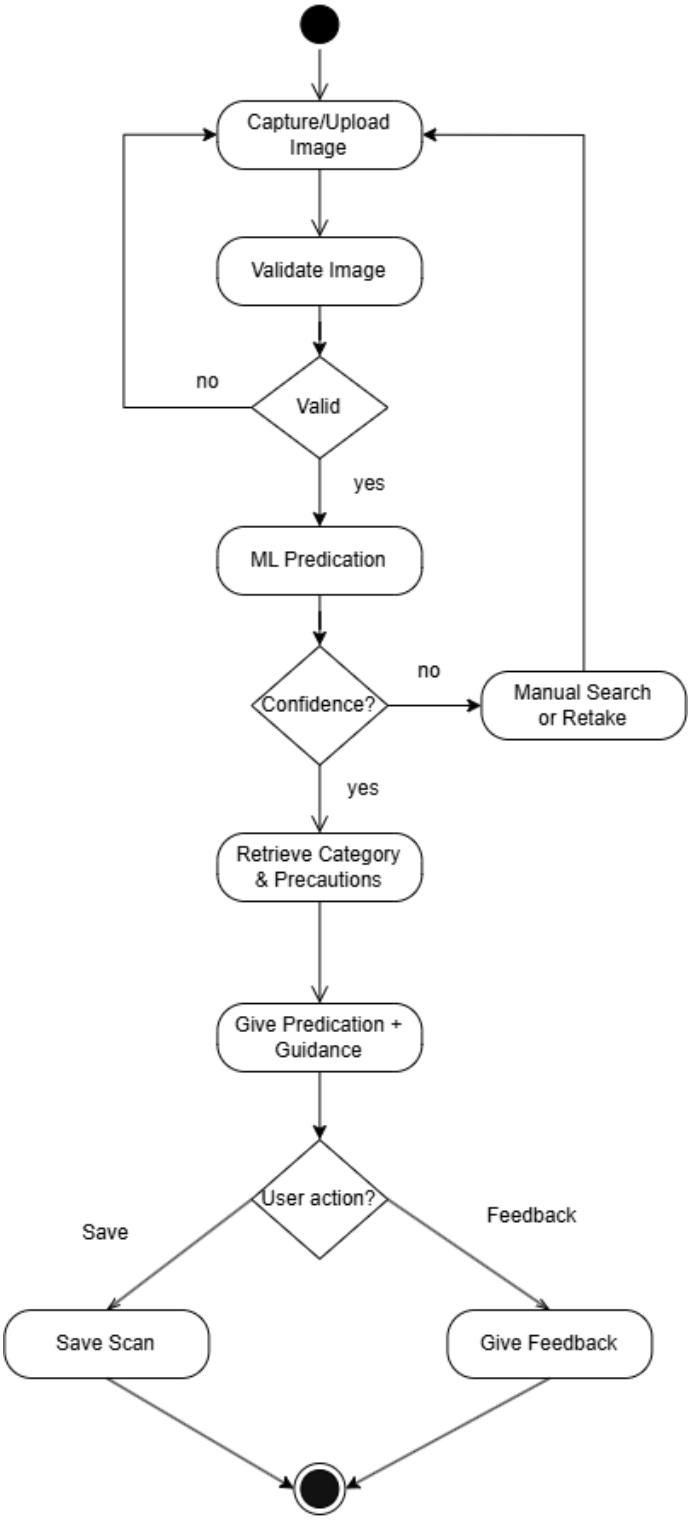


Figure 2.2: Activity Diagram for the Sortify Workflow

2.3 Use Cases Represented in the Prototype

The implemented prototype currently represents the standard user flow and an open demo administrator flow. Users can search the guide, run a simulated scan, inspect disposal guidance, save results, review history, ask the assistant, manage local profile data, and submit corrections. The admin screen supports catalog edits, feedback review, and prototype usage summaries.



Figure 2.3: Use Case Diagram for Sortify

2.4 Data Model Reference

The prototype uses local application state rather than a production database. The data represented in the app includes waste items, scans, feedback, profile data, assistant messages, and the selected location. This aligns with the entities expected for a fuller system while remaining limited to device-local demo storage.

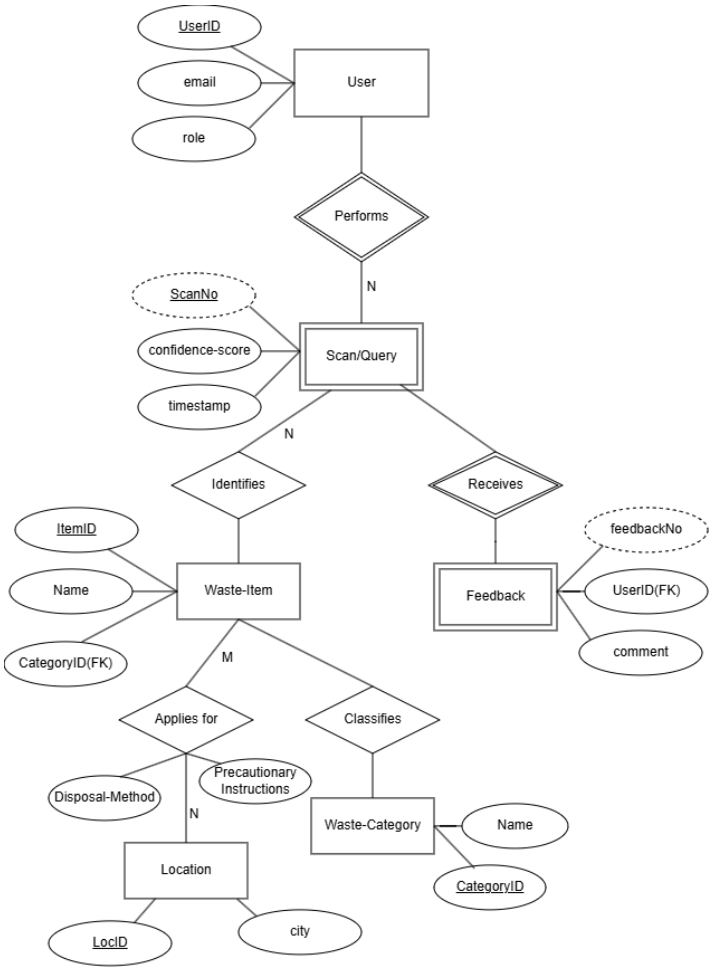


Figure 2.4: Entity Relationship Diagram for Sortify

2.5 Technical Stack Implemented

The implemented demo is built as an Expo/React Native application using TypeScript. It uses Expo Router for navigation, Expo Image Picker for camera and image-library access, React Native components for the interface, and AsyncStorage for local persistence. The current prototype is frontend-only and does not include a connected FastAPI backend, PostgreSQL database, PyTorch model, or live LLM-based assistant.

Chapter 3

Implemented Prototype Features

3.1 Application Navigation and Layout

The application has routed screens for Home, Waste Guide, Scan, Activity, History, Assistant, Learn, Profile, Admin, and Result. On iOS, the app uses native tabs. On other platforms, it uses a custom shell around the routed screens.

The Home screen greets the user when a local demo profile exists, provides a primary action to scan an item, and shows progress summaries and milestone information based on locally saved scans.

3.2 Waste Catalog and Guide

The prototype includes a structured local catalog of waste items. Each catalog entry stores an item name, waste category, material, description, disposal steps, optional caution text, and searchable keywords.

Implemented guide functionality includes:

- Searching by item name, material, or keyword.
- Filtering by category: recyclable, organic, general waste, e-waste, and hazardous.
- Pagination of catalog results.
- Opening an item result page from the guide.

The initial catalog includes examples such as plastic bottle, banana peel, cardboard box, household battery, glass jar, aluminium can, vegetable scraps, used tissue, snack wrapper, phone charger, old headphones, expired medicine, newspaper, dry leaves, and broken ceramic mug.

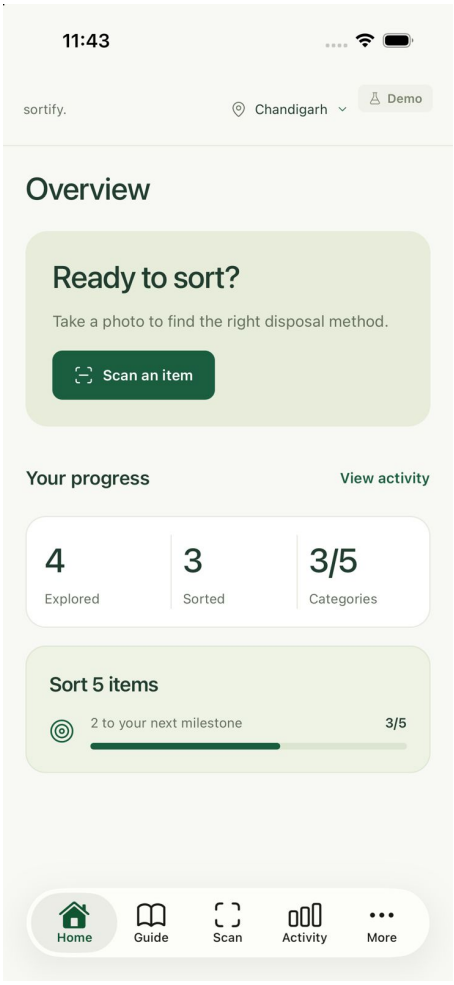


Figure 3.1: Sortify home screen showing the primary scan action and locally tracked progress.

3.3 Scan Demo

The scan screen implements two modes: photo and sample. In photo mode, the app can request camera permission, open the camera on supported platforms, open the image library, preview a selected image, reject images larger than 10 MB, and reject images smaller than 100 by 100 pixels.

The current identification step is simulated. The app lets the user choose a demo item and then opens the result screen after a short delay. The selected sample determines the result, and the app explicitly states that photos stay on the device.

Figure 3.2 shows the photo and sample entry points available in the scan interface.

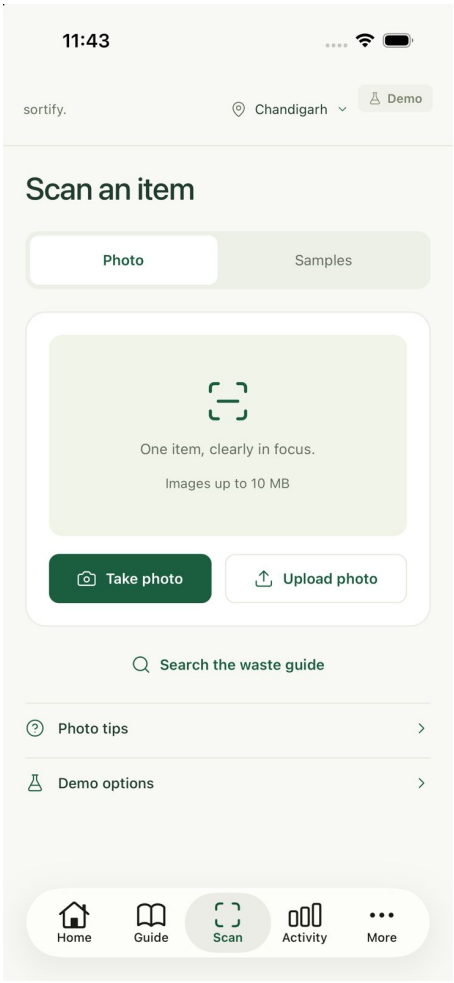


Figure 3.2: Scan screen with photo capture, upload, sample mode, and links to supporting guidance.

The scan demo also includes two controlled scenarios:

- Low-confidence result, which sends a 46% demo confidence value and asks the user to confirm before saving.
- Service unavailable, which shows an error state and suggests retrying or using the waste guide.

3.4 Result and Disposal Guidance

The result screen shows the selected item name, material, category badge, and demo confidence when available. It displays numbered disposal steps and caution notes for items that require extra care.

Implemented result actions include:

- Save the item to local history.
- Mark the item as sorted.
- View local guidance text based on the selected location and waste category.
- Ask the assistant about the current item.
- Submit a correction when the result appears wrong.

The location guidance is implemented as generated local text. It does not connect to verified local schedules or collection-point services.

3.5 History and Activity

The history screen lists saved scans and supports searching, filtering by sorted status, pagination, opening a saved result, and removing an item from history. Saved scans store item ID, date, confidence, source, and sorted status.

The activity screen summarizes locally saved activity. It includes a seven-day saved-item bar view and a category breakdown using the waste categories in the catalog. These summaries are based only on local demo data.

3.6 Profile and Local Data Settings

The profile screen implements a local demo account flow. Users can create or enter a demo account using name, email, and a password field. The password is validated for length during the demo flow but is not stored or verified. The profile stores only the user's name and email locally.

The data and privacy section explains the local storage behavior and provides a reset action that clears local activity and restores the original guide data.

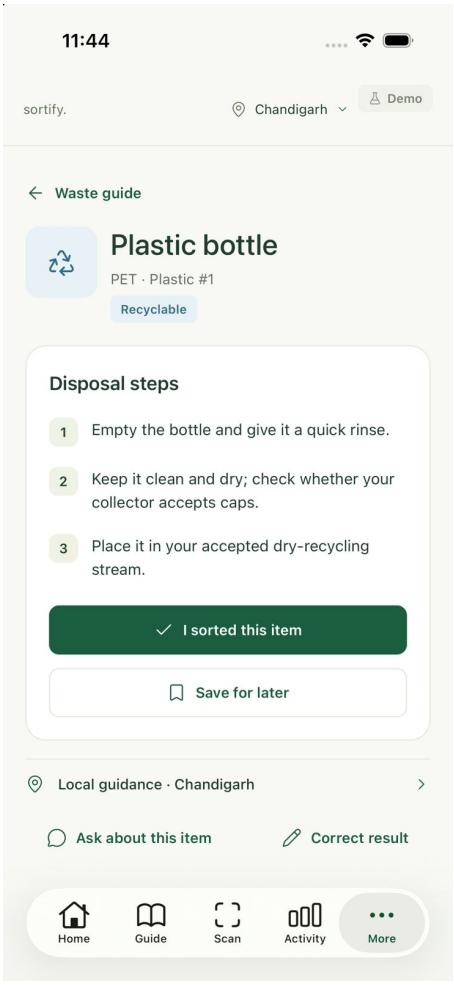


Figure 3.3: Waste guide item details for a plastic bottle, including disposal steps and user actions.

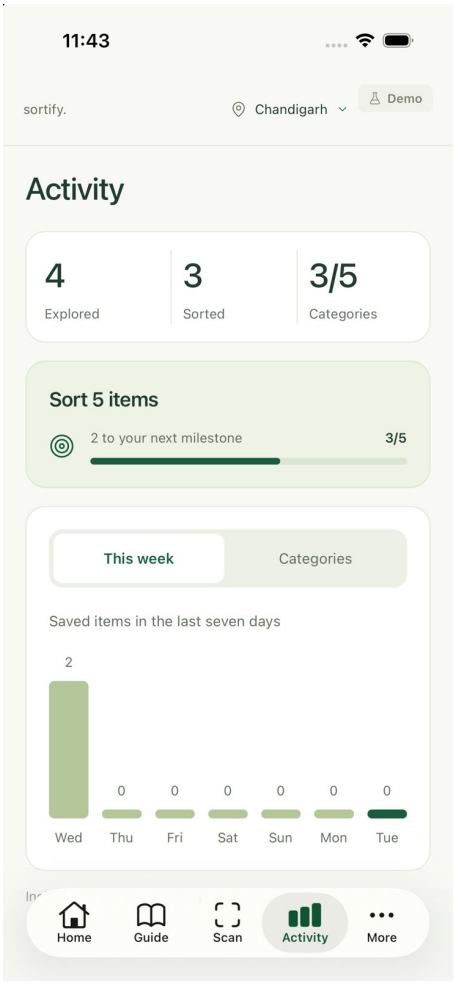


Figure 3.4: Activity screen showing explored and sorted item counts, milestone progress, and a weekly activity chart.

3.7 Scripted Assistant

The assistant screen implements a catalog-grounded scripted demo. Users can ask item-related questions, and the app matches words against catalog keywords. If it finds a catalog item, it returns the item category, disposal steps, caution note if present, and a reminder to check accepted materials locally.

The assistant also supports item context from a result screen, follow-up-style questions, conversation persistence in local storage, and clearing the local conversation. It is not connected to a live LLM or RAG backend in the current prototype.

3.8 Admin Demo

The admin screen is implemented as an open-access demo area. It includes three sections:

- **Waste catalog:** search catalog entries, add a new item, edit existing guidance, select a category, edit three disposal steps, add an optional safety note, and delete an item.
- **Feedback:** review corrections submitted from result screens and toggle their status between pending and reviewed.
- **Analytics:** view local counts for waste items, saved discoveries, feedback to review, demo predictions saved, manual selections saved, corrections submitted, and corrections reviewed.

The analytics section explicitly avoids fabricating model-performance values. It notes that accuracy, F1 score, inference latency, and confusion matrix results require a trained model and held-out evaluation set.

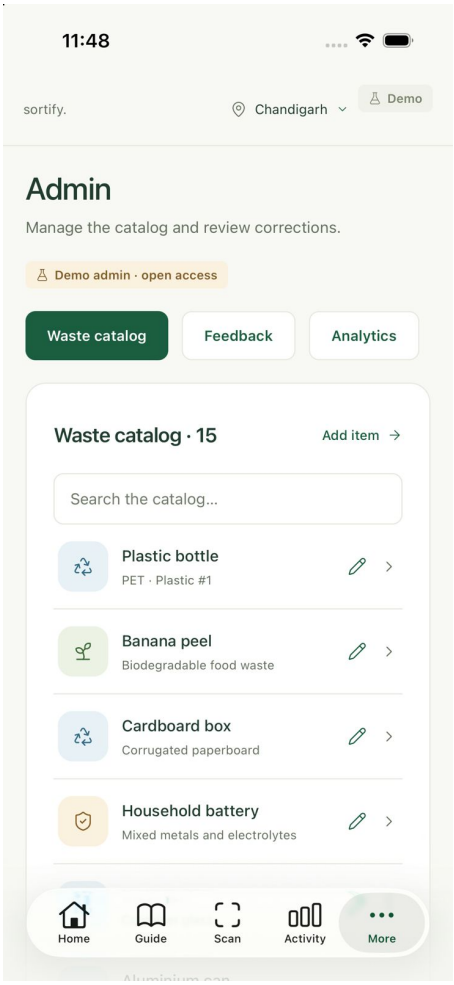


Figure 3.5: Admin demo showing the searchable waste catalog and catalog-management controls.

Chapter 4

Results and Evaluation

4.1 Prototype Status

The current implementation successfully demonstrates the frontend behavior of the core Sortify experience. The demo proves the app structure, navigation, catalog-driven guidance, local persistence, history flow, correction flow, assistant mockup, and admin catalog-management workflow.

4.2 Implemented Versus Not Yet Implemented

4.3 Current Evaluation Limits

The prototype is not ready for ML performance evaluation because it does not include a trained model, inference service, held-out dataset, or real prediction pipeline. Therefore, no accuracy, F1 score, confusion matrix, or inference-latency claims are made in this report.

The meaningful evaluation at this stage is functional: whether the implemented demo screens allow a user to complete the local scan-to-guidance workflow, search the catalog, save and review results, submit corrections, and manage catalog entries through the demo admin interface.

Area	**Prototype status**
Mobile app UI	Implemented as an Expo/React Native TypeScript demo.
Image capture/upload	Implemented through Expo Image Picker with basic image validation.
ML prediction	Simulated using selected demo samples and confidence values. No trained model is connected.
Waste guide	Implemented as a local searchable and filterable catalog.
Disposal guidance	Implemented through catalog disposal steps, caution notes, and generated local guidance text.
History and activity	Implemented using local scan records and local summary views.
Feedback	Implemented locally through correction submission and admin review status.
Assistant	Implemented as a scripted keyword-based catalog demo, not live AI.
Admin dashboard	Implemented as an open demo for catalog editing, feedback review, and local analytics.
Backend, database, deployment	Not implemented in the current prototype.

Table 4.1: Prototype implementation status

Chapter 5

Conclusion

5.1 Summary of Work

The prototype stage delivers a functional local mobile demo for Sortify. It implements the frontend flow for scanning an item through simulated predictions, viewing disposal guidance, searching the waste guide, saving history, tracking activity, using a local demo profile, asking a scripted catalog-based assistant, submitting corrections, and managing guide entries in an admin demo.

The implemented work remains aligned with the proposal's emphasis on actionable disposal guidance. At the current stage, the app demonstrates the user experience and data flow, while backend integration, trained ML inference, production authentication, a real database, live location services, and a live conversational AI backend remain outside the implemented prototype.

5.2 Next Work Based on Proposal Scope

The next implementation steps should focus on replacing simulated parts with real services: connecting a FastAPI backend, integrating a trained image-classification model, moving local catalog and scan records to a proper database, adding authenticated user accounts, and validating the system with representative waste items.